



LAB 4

LOOPS

Recommended Exercises

- **4.1.1**
- **4.1.2**
- **4.1.6**
- **4.2.3**

Exercise 4.1.1

- Read request from lab text

Exercise 4.1.1 (cont'd)

- We read one number at a time with the `input()` function
- Reading stops when the user presses enter without entering any characters

Exercise 4.1.1 (cont'd)

■ Suggestions:

- For request I , the partial sum will be printed after each acquisition.
This could be an example of I/O in your program:

```
Enter a number: 5
Partial sum: 5
Enter a number: 7
Partial sum: 12
Enter a number: 2
Partial sum: 14
...etc...
```

- Request II requires that the maximum and minimum be **calculated together** .
NB: Initially, the first number read will be **both the maximum and the minimum** (see examples done in class)

Exercise 4.1.2

- Read request from lab text

Exercise 4.1.2 (cont'd)

■ Suggestions:

- For some points, it is convenient to use a **for loop** to iterate over the **characters** of the read string. For others, the:

```
for i in range()
```

scheme is more convenient , which allows you to access the **position** of an element.

Exercise 4.1.2 (cont'd)

■ Suggestions:

- Point a requires you to print only the **characters that match a given condition** . You can decide to:
 - [PREFERABLE]
Print the matching characters **directly inside the loop** (but in this case pay attention to the options of the `print()` function !)
 - **Create a new string** containing only the desired characters inside the loop, and print it after the loop ends

Exercise 4.1.2 (cont'd)

■ Suggestions:

- Point **b** requires printing only the characters **whose position (index)** satisfies a given condition...
 - Note the difference (and similarities) with the previous point
- Point **c** is similar to **point a** , with the only difference that the characters that match the condition must be replaced with an underscore (`_`), not deleted (**and strings are immutable...**)
- Point **d** requires **counting** the number of occurrences of characters belonging to a given set. We will therefore need a **counter variable** (scheme seen in class).

Exercise 4.1.2 (cont'd)

- **Suggestions:**

- Point **e** again requires acting on the character indices (printing them, in this case), and is therefore more similar to **point b** .

Exercise 4.1.6

- Read request from lab text

Exercise 4.1.6 (cont'd)

- Read a number (`limit`)
- For all values `number` between 2 and `limit` included
For each `factor` value between 2 and `num`
If `num` is divisible by `factor`, then it is not prime
- NOTE: You can stop at `num/2` (instead of `num`) because *dividing a number by a value greater than its half will always produce a result with a fractional part.*
 - Further optimization: actually you could stop at the integer closest to `sqrt(N)`

Exercise 4.2.3

- Read request from lab text

Exercise 4.2.3 (cont'd)

Pseudocode :

Read the initial velocity

*$s = v * \Delta T$*

While $s > 0$:

if the current time is multiple of 1 second:

print s_exact and s

update s

update v

update time

NOTE : *I must only print once every second, **not every time***

NOTE2 : *Can I overwrite all values at each iteration or do I have to save some?*

Tip : The debugger allows you to stop the program execution even inside the loops! You can use it to stop the program after each simulation step